

SDLC 2.0

The Agentic Shift

Agile Network India | Chennai Chapter

22 August 2026 | RMZ Millenia, Perungudi

Meet Chintu:

An Agent That Learned Our Data Lineage

What agentic AI actually costs in an enterprise — and the four walls we hit

Sarveshwaran R

Senior Research Scientist, NIQ | 13 years in enterprise AI & data engineering

It Started With One Question in a Review

A senior leadership director, in a routine platform review

"Which source systems feed our churn model?"

Reasonable question. Senior person. Room full of people who had built the thing.
And not one of us could answer it with confidence.

It was documented

We had lineage documentation. Written carefully, by good engineers.

It had drifted

Dozens of changes had landed since. The document described a system that no longer existed.

The truth was in people

The only accurate lineage lived in two architects' heads. And memory does not scale, or attend meetings.

Let Me Explain This the Way I Explain It at Home

For the next forty minutes, our data platform is a restaurant. Everything else follows from that.

IN THE KITCHEN		IN OUR PLATFORM
The restaurant	≡	Our data platform
The signature dish	≡	The churn model
Ingredients, and which farm they came from	≡	Source tables
Ingredient traceability	≡	Data lineage
The recipe book on the shelf	≡	Our documentation
The head chef	≡	The principal architect
Cost per plate	≡	Cost per pipeline

The Recipe Book Was Fiction

What the documentation said

Six source tables.

Written eighteen months ago.

Signed off. Filed. Never opened again.

Technically information. Historically accurate.

What was actually running

Eleven source tables.

Two of them created last quarter.

One upstream job quietly deprecated.

Currently fiction.

Documentation is a snapshot. A data platform is a movie.

1,400

pipelines in scope

~3 hrs

to trace one properly, by hand

4,200

engineer-hours to do it once

Then

start again, because it changed

What My Team Tried First

We are an AI team. So of course our first instinct was: throw it at a model.

The two-hour prototype

1. Read the entire SQL file
2. Paste all of it into the prompt
3. Ask for the lineage in plain English
4. Print whatever comes back

*No chunking. No output schema. No routing.
No cost tracking. Biggest model, every time.*

And it worked.

- ✓ Correct source tables, first attempt
- ✓ Readable transformation descriptions
- ✓ Caught a PII column we had missed
- ✓ Two hours, start to finish

The demo went down very well indeed.

This is where most AI talks end. This is where ours actually began.

Everything after this slide is the part nobody demos.

Then Four Walls Appeared

WALL 1

The Bill

Our cost per plate came out higher than simply hiring people to do the tracing.

WALL 2

The Kitchen

We sent 1,400 orders in at once. The kitchen stopped taking orders at 300.

WALL 3

The Recipe

"Our production SQL is not going to a third-party model." End of conversation.

WALL 4

The People

Leadership wanted a number. The team wanted to know if they were being replaced.

Only one of these four walls was a technology problem.

Cost Per Plate

What we projected, per month

~\$8,200

1,400 pipelines, re-run whenever one changed,
biggest model, long prose output

What it was replacing

~\$5,600

The equivalent manual tracing effort
at a loaded engineering rate

Our AI solution cost more than the problem it was solving.

Why leadership paused it in week three

Not because it failed. Because nobody in the room could explain the cost per plate — and a recurring bill that nobody can explain is a governance problem long before it is a finance problem.

So We Learned What We Were Actually Paying For

INPUT TOKENS

Everything you send

Prompt, schema and the entire SQL file, on every single call. Cheap per token. You send an enormous number of them.

OUTPUT TOKENS

Everything it writes

Typically several times the price of input. Ask for a long explanation and you pay for a long explanation.

FREQUENCY

How often you call

The multiplier everyone forgets. 1,400 pipelines once is a very different bill from 1,400 on every commit.

One lineage extraction, measured

	Our first attempt	After we fixed it
Input tokens	~12,400	~2,800
Output tokens	~3,100	~900
Model used	Largest, always	Routed by complexity
Cost per pipeline	~\$0.42	~\$0.04

What Actually Cut the Bill

THE BIGGEST SINGLE WIN

Before: "Explain the lineage in detail."

Three paragraphs back. We used one line of it.

After: "Return ONLY this JSON schema."

Goes straight into the catalog. Same shape every run.

3,100 → 900

output tokens per pipeline. Accuracy did not move.

We were ordering full meals and eating only the rasam.

2

Cap the response length

A runaway response is a runaway invoice. Set a hard ceiling on every call.

3

Cache the fixed part

Our schema and instructions were identical on all 1,400 calls. Only the SQL changed.

4

Cook it overnight

Nobody needed pipeline 900 traced in two seconds. Trade speed for a discount.

5

Send only what is needed

Parse the file first. Send the joins and the select list, not 400 lines.

Not one of these five is an AI technique. This is ordinary engineering discipline, applied to a new kind of bill.

The Night the Kitchen Stopped Taking Orders

What we did

Sent all 1,400 orders into the kitchen at the same time, because the work was parallel and we were feeling confident.

What happened

429 Too Many Requests. Our retries made it worse. The job died at 300, having paid for calls that returned nothing.

The limits that actually bind

TPM Tokens per minute — the one that got us

RPM Requests per minute

Concurrency How many orders in the pass at once

Context window A per-dish limit, not a rate

What fixed it

A queue, with a limit

Only so many orders in the pass at once, tuned to sit under the ceiling.

Back off, politely

With jitter. Hammering a busy kitchen harder does not get you served faster.

Remember where you stopped

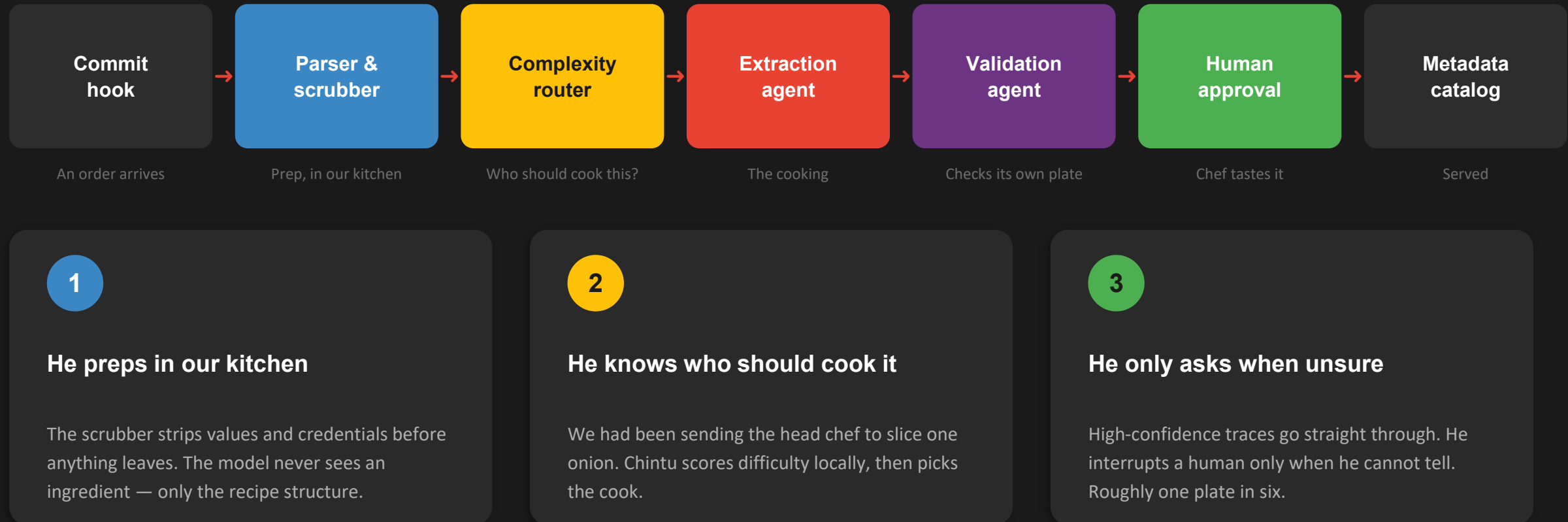
Resume at 300. Never pay twice for the first 299.

Cook off-peak

Run the backfill overnight. Keep daytime capacity for real users.

So We Stopped Prompting, and Built Chintu

Chintu is not a chatbot. He is a kitchen assistant we trained ourselves — and he only ever bothers the head chef when he genuinely does not know.



Chintu is not clever. Chintu is disciplined. That turned out to be the more useful quality.

What Chintu Delivered

~\$0.04

cost per plate

Down from ~\$0.42

11 days

to clear the backlog

Against 4,200 manual hours

~1 in 6

plates need the chef

The rest go straight through

0

ingredients sent out

Structure only, by design

94%

of traces approved with no edit

Accuracy is a model metric. Approval rate is an adoption metric. Only one of them tells you the system is actually being used.

The thing we did not expect

Chintu found lineage that no document had ever captured, and one upstream table that had been feeding a live model after everybody believed it was retired. Nobody asked him to look for that.

LIVE DEMO — Chintu, running

"Our Data Is Not Going to an LLM"

The sentence that ends most enterprise AI projects

And security was right. We had been sending production SQL — real table names, real business logic — to an external endpoint, and not once had we asked whether we were allowed to.

The reframe that unlocked it

1

It never needed the ingredients

Traceability comes from the recipe, not from the food. Chintu reads structure. Not one row of customer data was ever required.

2

Prep before it leaves

Literal values, credentials and sample rows are stripped by a pre-processor inside our own network.

3

Security picks the kitchen

Private endpoint, zero-retention terms, or self-hosted. That decision was theirs to make, not mine.

You can show someone the recipe without showing them the guest register.

What Actually Crosses the Pass

STAYS IN OUR KITCHEN

- ✓ Production databases
- ✓ Customer records and PII
- ✓ Literal values in queries
- ✓ Credentials and connection strings
- ✓ The parser and pre-processor
- ✓ The approved lineage catalog

GOES TO THE MODEL

- Table and column names
- Data types
- Join and CTE structure
- Transformation expressions

**Zero rows. Zero customer data.
Structure only.**

This one diagram did more for approval than every business case I wrote.

The Leadership Conversation

WHAT I SAID THE FIRST TIME

"We can use an LLM to automatically extract data lineage from our SQL pipelines."

What they actually heard

- ✗ A recurring cost with no ceiling
- ✗ A new vendor dependency
- ✗ A compliance question they now own
- ✗ Another initiative that may not land

Paused in week three.

WHAT I SAID THE SECOND TIME

"We can clear a 4,200-hour tracing backlog for about four cents a pipeline, with nothing leaving our network."

What changed

- ✓ A known cost per plate, not an open bill
- ✓ A named business outcome
- ✓ The data question answered upfront
- ✓ A measurable success criterion

Approved in one meeting. Then showcased twice.

Same project. Same technology. Different sentence.

What This Did to Our SDLC

SDLC 1.0

Before

- Lineage traced by hand, or not at all
- The recipe book goes stale on the next commit
- Audit answered by reading code
- Change avoided, because impact unknown

SDLC 1.5

AI-assisted

- An engineer runs extraction when asked
- AI drafts, engineer edits and approves
- Still a manual trigger, still ad hoc
- Cost visible, but only per run

SDLC 2.0

Chintu

- The commit itself triggers the trace
- Chintu routes, extracts and self-checks
- A human approves only what he flags
- The recipe book cannot go stale

The technology changed once. The process changed three times.

What Changed for the Team

"People don't resist AI. They resist uncertainty."

? Am I being automated out of a job?

? Who signs off if Chintu gets it wrong?

? Do I have to learn LLMs now?

? Will this be abandoned in six months?

What actually moved them

The chef still tastes the plate

Chintu drafts. An engineer approves. Nothing reaches the catalog without a human name against it.

We took away the worst job

Nobody enjoyed manual lineage tracing. We did not automate the interesting part of anyone's work.

They wrote Chintu's rules

The people who knew the pipelines wrote the extraction logic. He became their assistant, not mine.

Nobody lost a job. The team did not shrink. They just stopped writing documents nobody read.

The Three Conversations You Will Have to Win

THE SPONSOR

"What will this cost me every month?"

Give them a cost per plate, not a monthly total. "Four cents per pipeline, and here is what drives that number." A unit cost can be forecast, budgeted and defended. A lump sum can only be feared.

SECURITY

"Where exactly does our data go?"

Draw the boundary diagram before they ask for it. Show what crosses the pass and what never does. If the honest answer is that sensitive data does cross, say so, and bring deployment options instead of an argument.

THE TEAM

"What happens to my job?"

Name the exact task that is going away and name who still signs off. Vagueness here always reads as bad news. "You stop writing lineage documents, you still approve every one" is a sentence people can work with.

I spent more hours in these three conversations than my team spent writing Chintu.

What Changed in My Head

FROM

Start with the model

Ask what AI can do

Biggest model, always

Measure accuracy

Ask forgiveness on data

Pitch the technology



TO

Start with the painful task

Ask what the task costs today

Cheapest cook who clears the bar

Measure approval rate and adoption

Draw the boundary before you build

Pitch the cost per plate

Want to build systems like Chintu?

Scan to join the free live session

Free Live Webinar



- Learn how to build AI agents step by step
- Work on real-world business use cases
- Transition into AI with practical skills

Socials

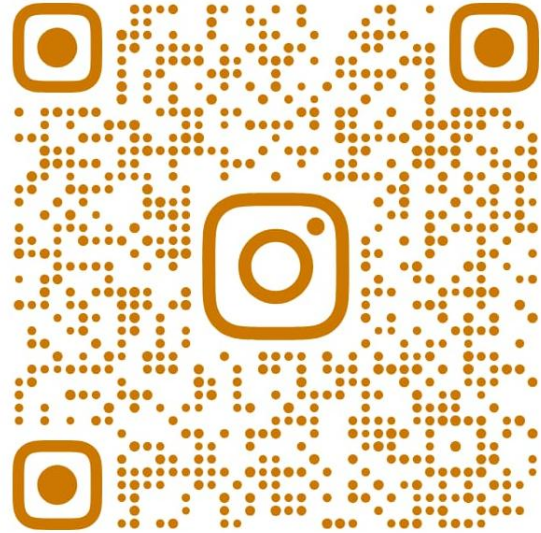


Sarveshwaran Rajagopal

Applied AI Practitioner | Founder
- Learn with Sarvesh | Speaker | Awar...



LinkedIn



@WITHSARVESH.AI

Instagram



WhatsApp

Chintu is not clever.

He is disciplined.

He knows what he costs.
He knows what he is allowed
to see. And he knows when
to ask a human.

That was the whole trick.

*The winners in the Agentic era
won't be the organisations with
the most AI tools.*

*They'll be the ones who know
what their AI costs, where their
data goes, and who signs off
when it gets it wrong.*

Sarveshwaran

Senior Research Scientist | 13 years in enterprise AI

Thank You | Questions?

Agile Network India | Chennai | 22 Aug 2026